PCB Defect Detection

Ahmed Aly Maged Mohamed Mohamed Sayed (Mat: 946984)
Askar Amro Emad Hassan (Mat: 946807)

Table of Contents

01 **Our Problem**

02 **Data Analysis**

Dataset and Analysis before training

03 **Solutions**

- Training a model from scratch
- Training a model with a ResNet backbone

04 **Alternative**

Using YOLOv11 with edited blocks

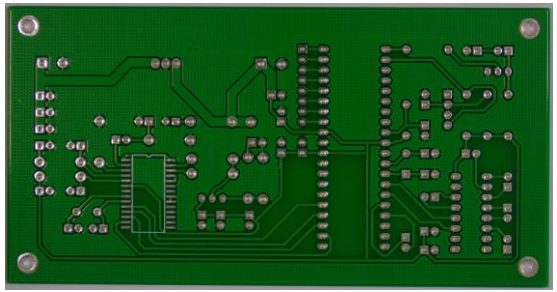
01

Our Problem

Our Problem

PCB defects are small, varied, and expensive to inspect manually. In an industrial setting, missing holes, mouse bites, open circuits, shorts, spurs, and spurious copper can reduce board quality, cause downstream failure, and slow production. The challenge is to build a model that can reliably detect and localize these defects from images.

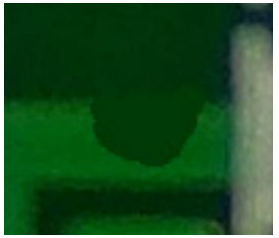
Our Problem



Missing Hole



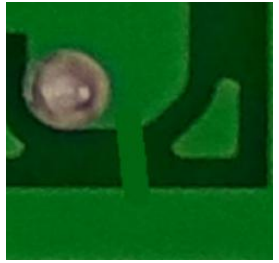
Mouse Bite



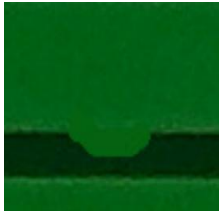
Open Circuit



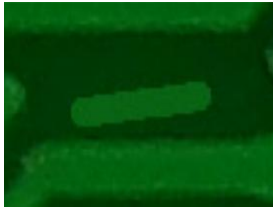
Short Circuit



Spur



Spurious Copper



02

Data Analysis

Dataset and Analysis before training

Dataset

About the dataset

- **693** PCB **images** total
- **2,953** annotated **defect** instances total (Average of 4 defects per image)
- **6** defect **classes**
- Each image is RGB, with **image sizes** ranging from 2240 x 1586 to 3056 x 2530

Class distribution by annotated defects

- Missing hole: 497
- Mouse bite: 492
- Open circuit: 482
- Short: 491
- Spur: 488
- Spurious copper: 503

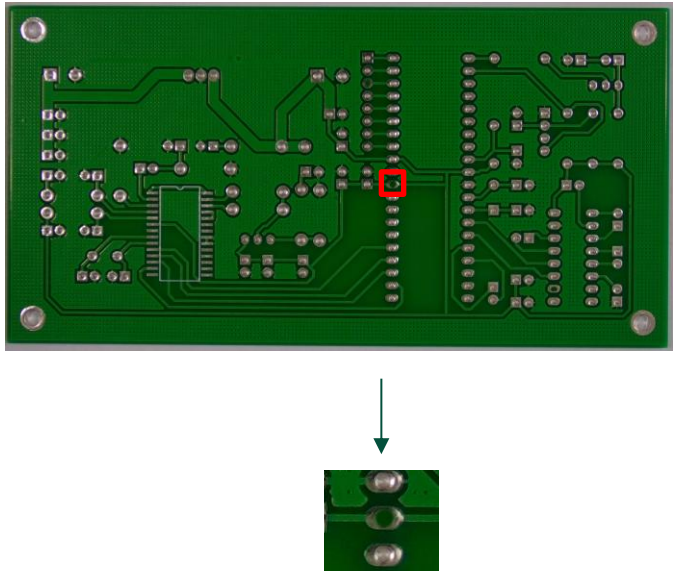
Dataset

Problem with the dataset

- Each image is RGB, with **image sizes** ranging from **2240 x 1586 to 3056 x 2530**

By class, the average width and height of defects are:

- Missing hole: 66.76 x 68.10
- Mouse bite: 63.07 x 61.43
- Open circuit: 52.36 x 49.79
- Short: 86.74 x 78.75
- Spur: 75.41 x 69.36
- Spurious copper: 80.21 x 77.48



Relatively too small !!
Hard for feature extraction and for detection

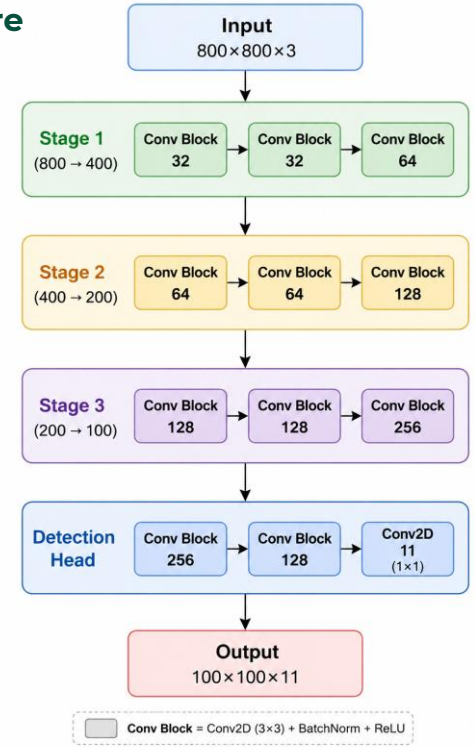
03

Solutions

Training a model from scratch
Training a model with a ResNet backbone

Training a model from scratch

CNN Architecture (Grid Yolo-Style)



Number of parameters: 1,654,507

Loss Function Design

$$L = L_{obj} + L_{cls} + 5L_{box}$$

Where

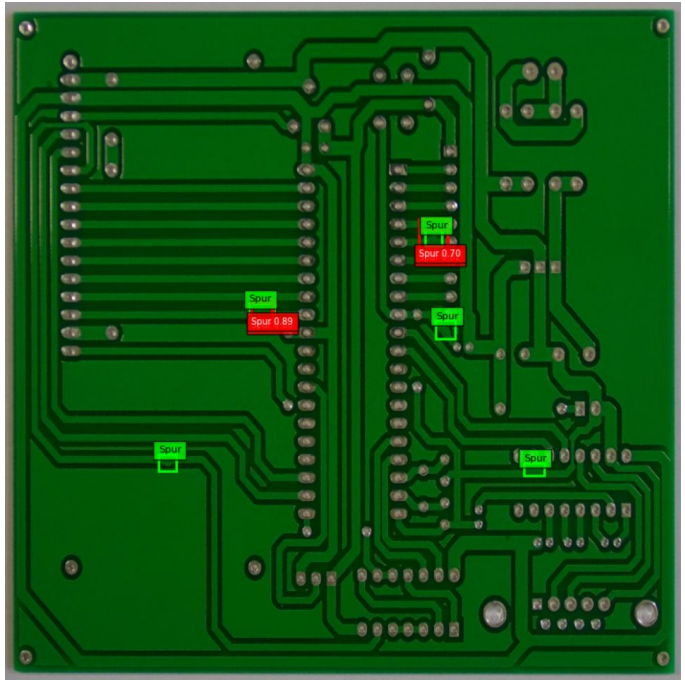
- Objectness Loss: BCEWithLogits
- Classification Loss: Cross Entropy
- Localization Loss: Smooth L1

Data Augmentation

- Horizontal Flip
- Vertical Flip
- Slight Brightness Adjustment
- Slight Contrast Adjustment

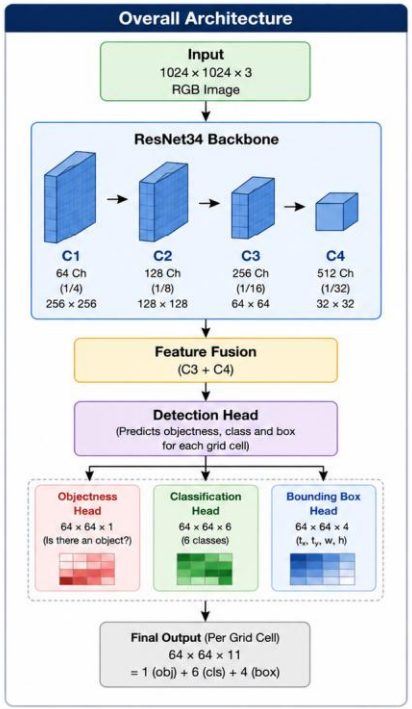
*Figure generated by ChatGPT

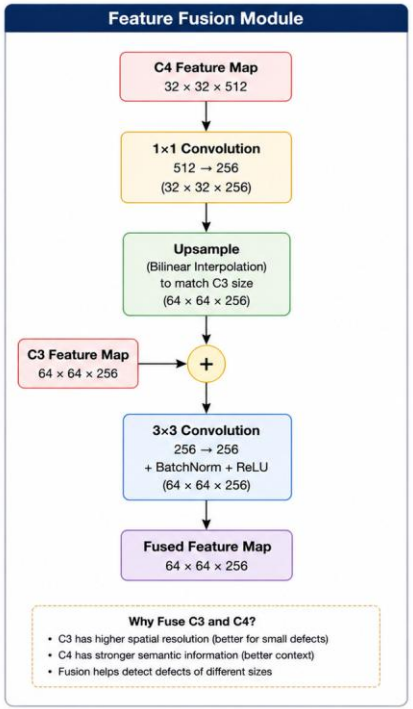
Training a model from scratch



Kinda Failed !!
The model wasn’t able to extract the features well due to the small size of the defects

Training a model with a ResNet backbone





Number of parameters: 22,893,899

Loss Function Design

$$L = L_{obj} + L_{cls} + L_{box}$$

Where

- Objectness Loss: BCEWithLogits
- Classification Loss: Cross Entropy
- Localization Loss: Smooth L1

Data Augmentation

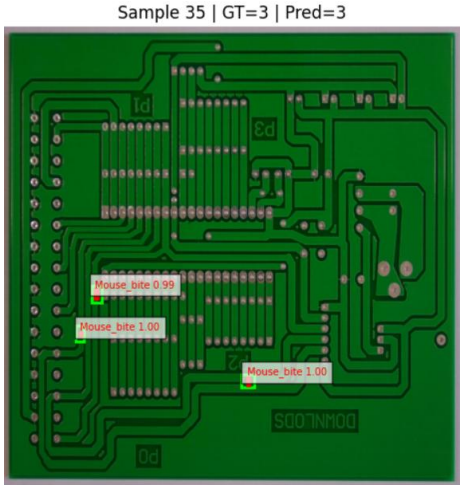
- Horizontal Flip
- Vertical Flip

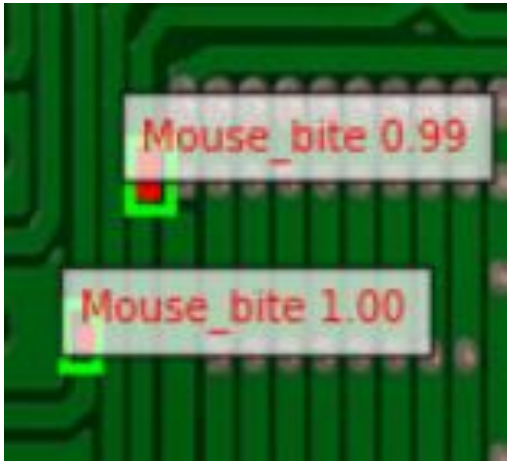
*Figure generated by ChatGPT

Training a model with a ResNet backbone

Metric	Value
Precision*	78.14%
Recall**	75.17%
F1	76.63%
Image Classification Accuracy***	95.71%

But why aren't precision and recall the best ?





For our goal and project, it's a success

* Precision = TP / FP+TP
** Recall = TP / FN+TP

***Image Classification Accuracy: How many correct defects are classified in the images despite the localization

04
Alternatives

Using YOLOv11 with added blocks

Training a pretrained YOLOv11 model

YOLOv11 already very powerful !!

Containing:

- SPPF (Spatial Pyramid Pooling – Fast): Multi-scale context extraction.
- C2PSA (Cross-Stage Partial with Spatial Attention): Attention mechanism that highlights important image regions.
- C3k2 (CSP Bottleneck with Two Small Kernels): Lightweight feature extraction block for faster and more efficient learning.

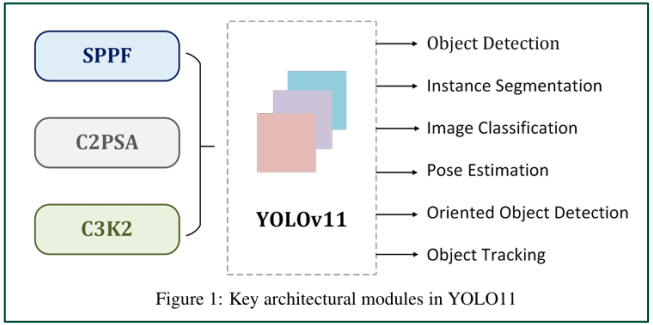


Figure 1: Key architectural modules in YOLO11

But still the same problem
Defects relatively too small as YOLO is trained on COCO

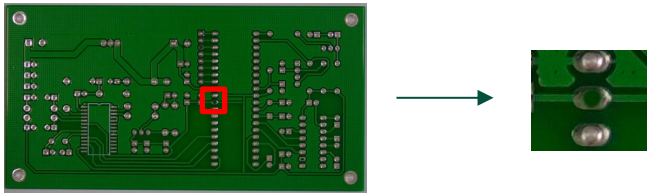


*Figure taken from the paper “YOLOV11: AN OVERVIEW OF THE KEY ARCHITECTURAL ENHANCEMENTS”

Training a pretrained YOLOv11 model enhanced with MC-VAM

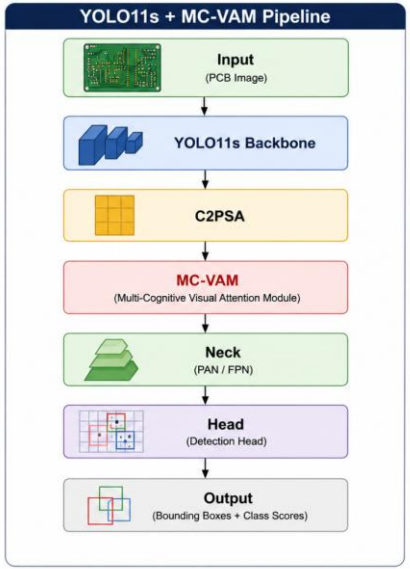
(inspired by the paper “A multi-cognitive PCB defect detection model integrating Mamba)

Multi-Cognitive Visual Attention Module (MC-VAM)

MC-VAM is a lightweight attention module designed to improve feature extraction by capturing information at multiple spatial scales.

Objectives

- Enhance small defect detection
- Capture local and global context simultaneously
- Preserve important shallow features
- Improve feature representation with minimal computational overhead



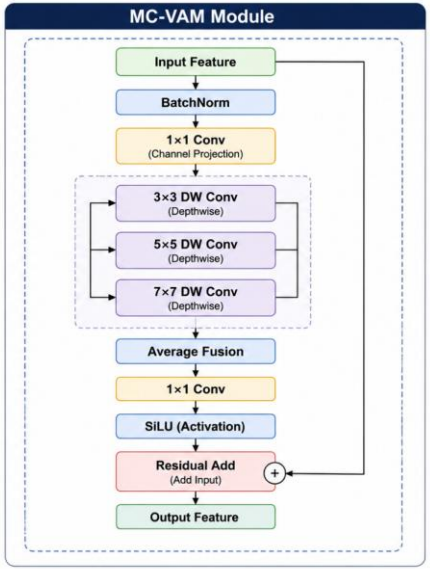


*Figure generated by ChatGPT

Comparing results

Metric	YOLOv11 baseline	Enhanced YOLOv11
Precision*	0.969	0.971
Recall**	0.947	0.946
mAP50***	0.967	0.972
mAP50-95****	0.529	0.527

* Precision = TP / FP+TP

** Recall = TP / FN+TP

*** mAP50 : Mean Average Precision at IoU = 50%. Measures overall detection performance.

**** mAP50-95 : Mean Average Precision averaged across IoU thresholds from 50% to 95%. Measures both detection and localization accuracy.

Thanks

Do you have any questions?

CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, and infographics & images by **Freepik**